



Chapter 4

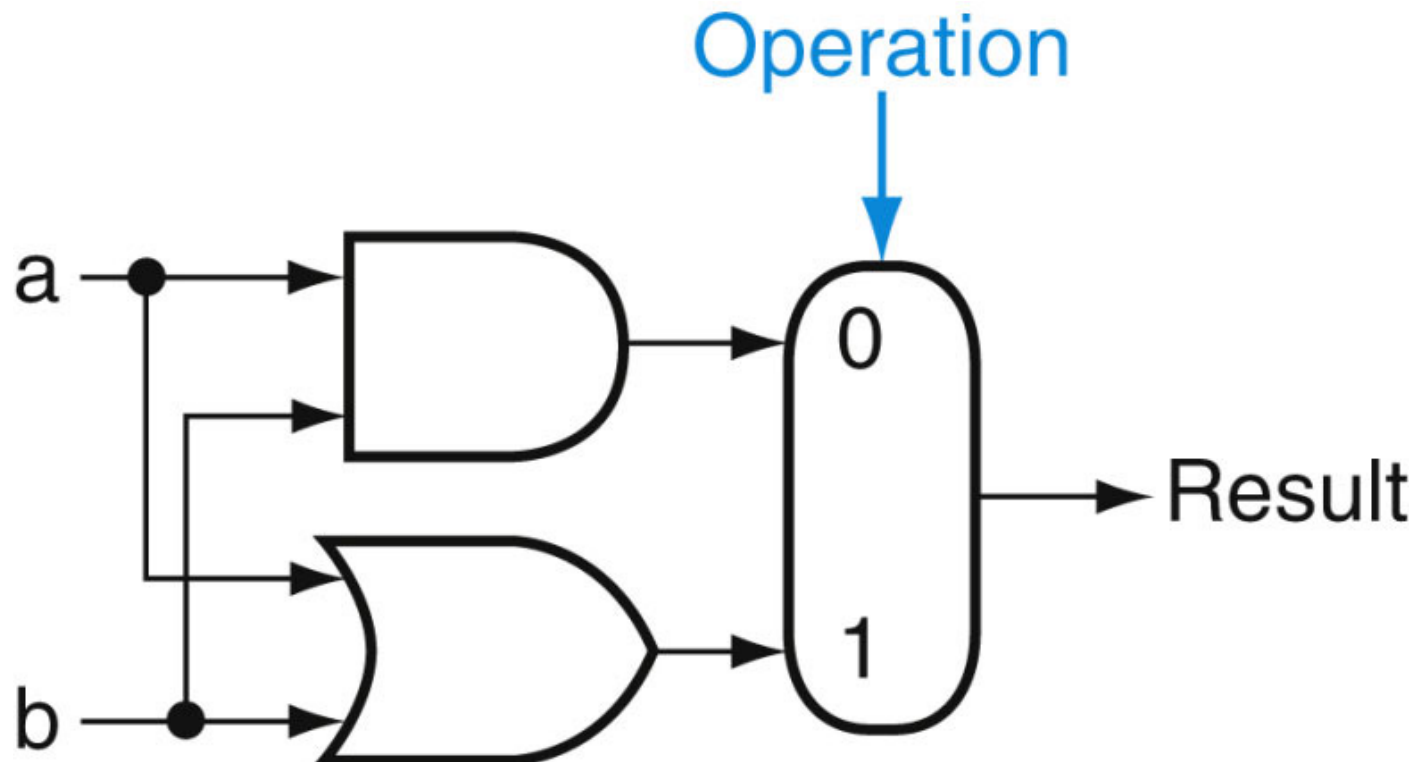
The Processor

Readings

- COD 5th edition
 - Chapter 4
 - Appendix A The Basics of Logic Design A.5 A.7 A.8
 - Appendix C Mapping Control to Hardware C.2 C.3

ALU

- 1-bit logical unit for **AND** and **OR**.
- The multiplexor selects a AND b or a OR b, depending on the value of Operation.

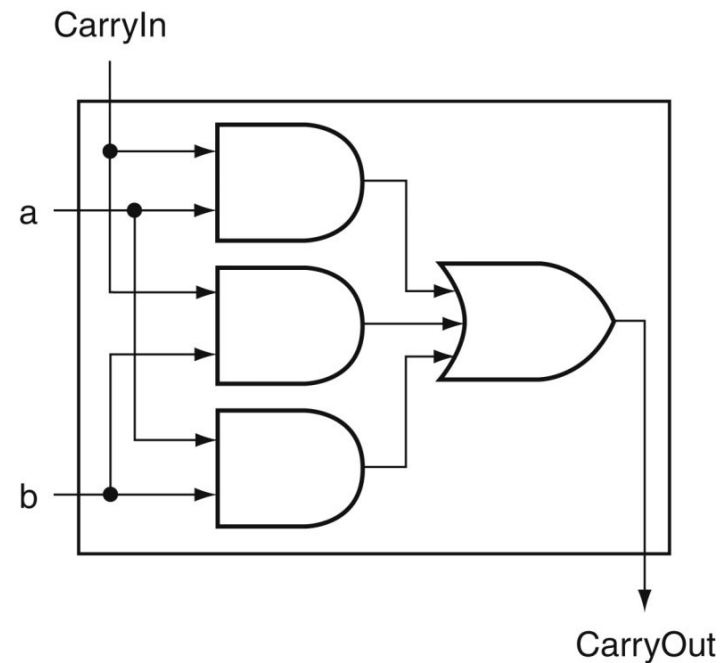
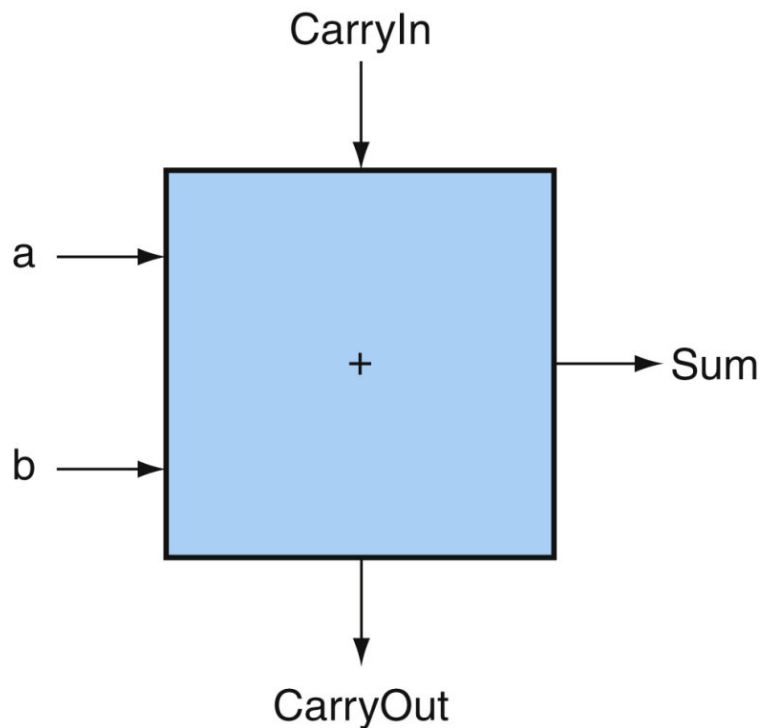


ALU

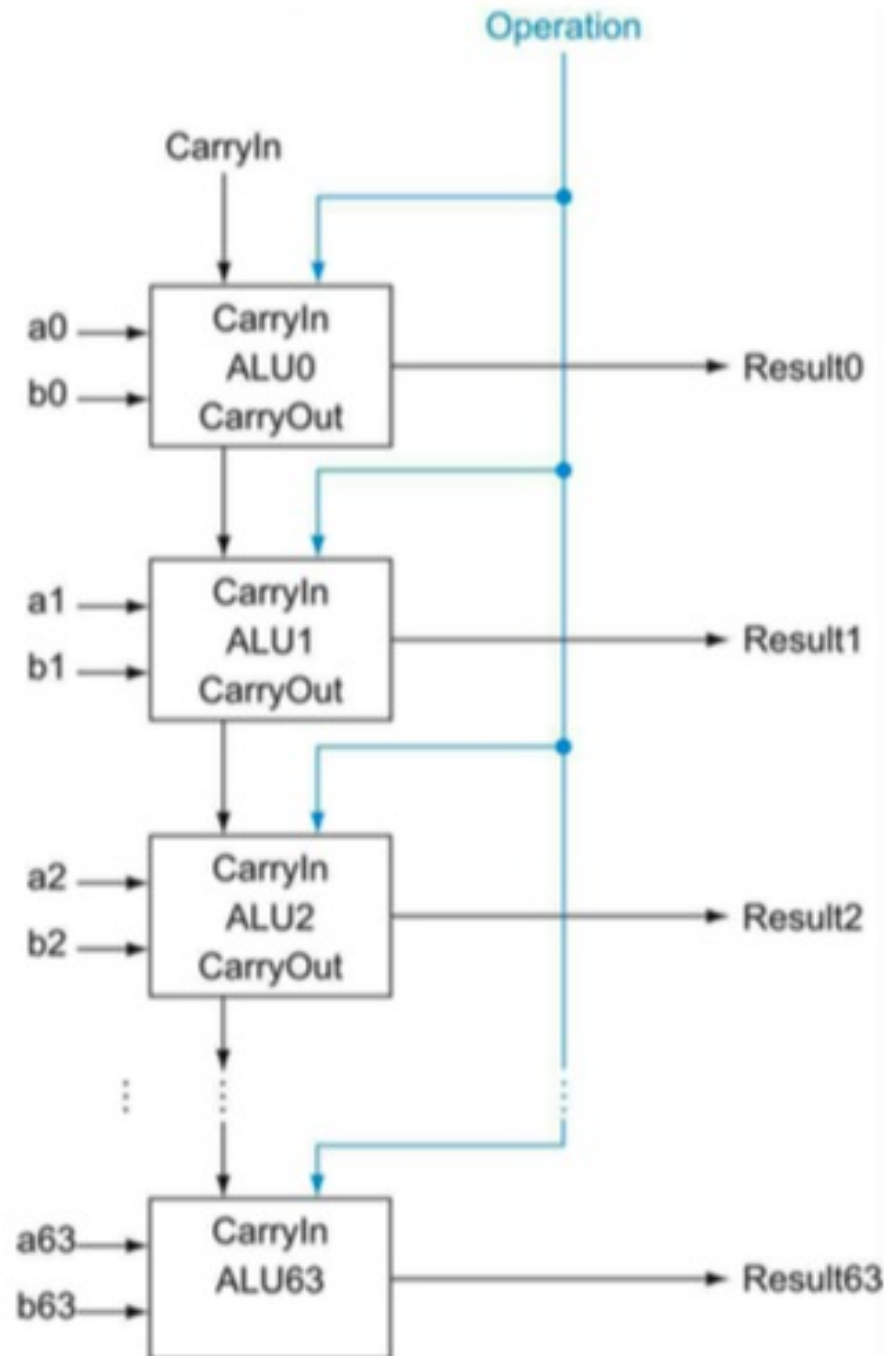
- 1-bit adder, a full adder, a (3,2) adder

$$\text{CarryOut} = (b \cdot \text{CarryIn}) + (a \cdot \text{CarryIn}) + (a \cdot b)$$

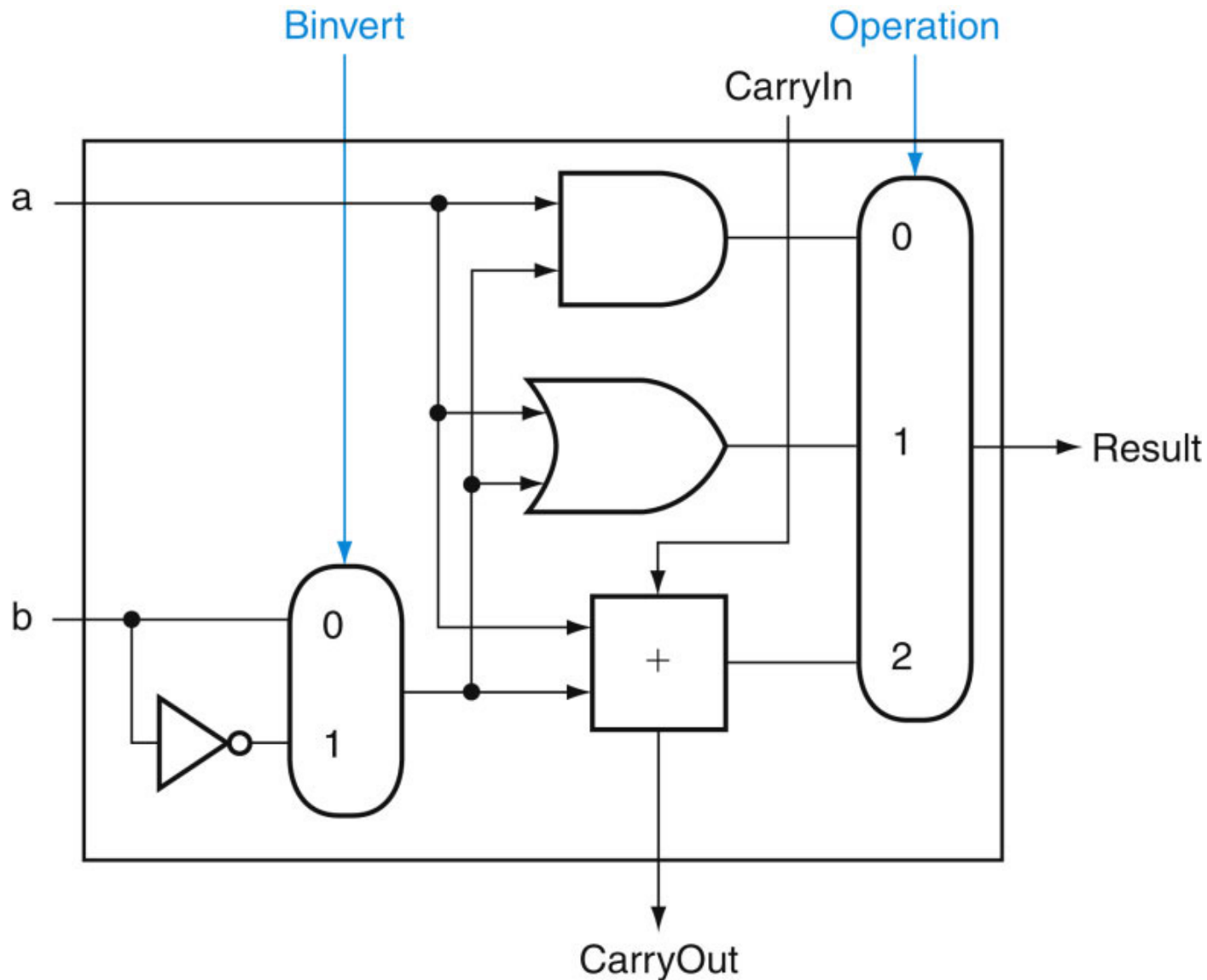
$$\text{Sum} = (a \cdot \bar{b} \cdot \overline{\text{CarryIn}}) + (\bar{a} \cdot b \cdot \overline{\text{CarryIn}}) + (\bar{a} \cdot \bar{b} \cdot \text{CarryIn}) + (a \cdot b \cdot \text{CarryIn})$$



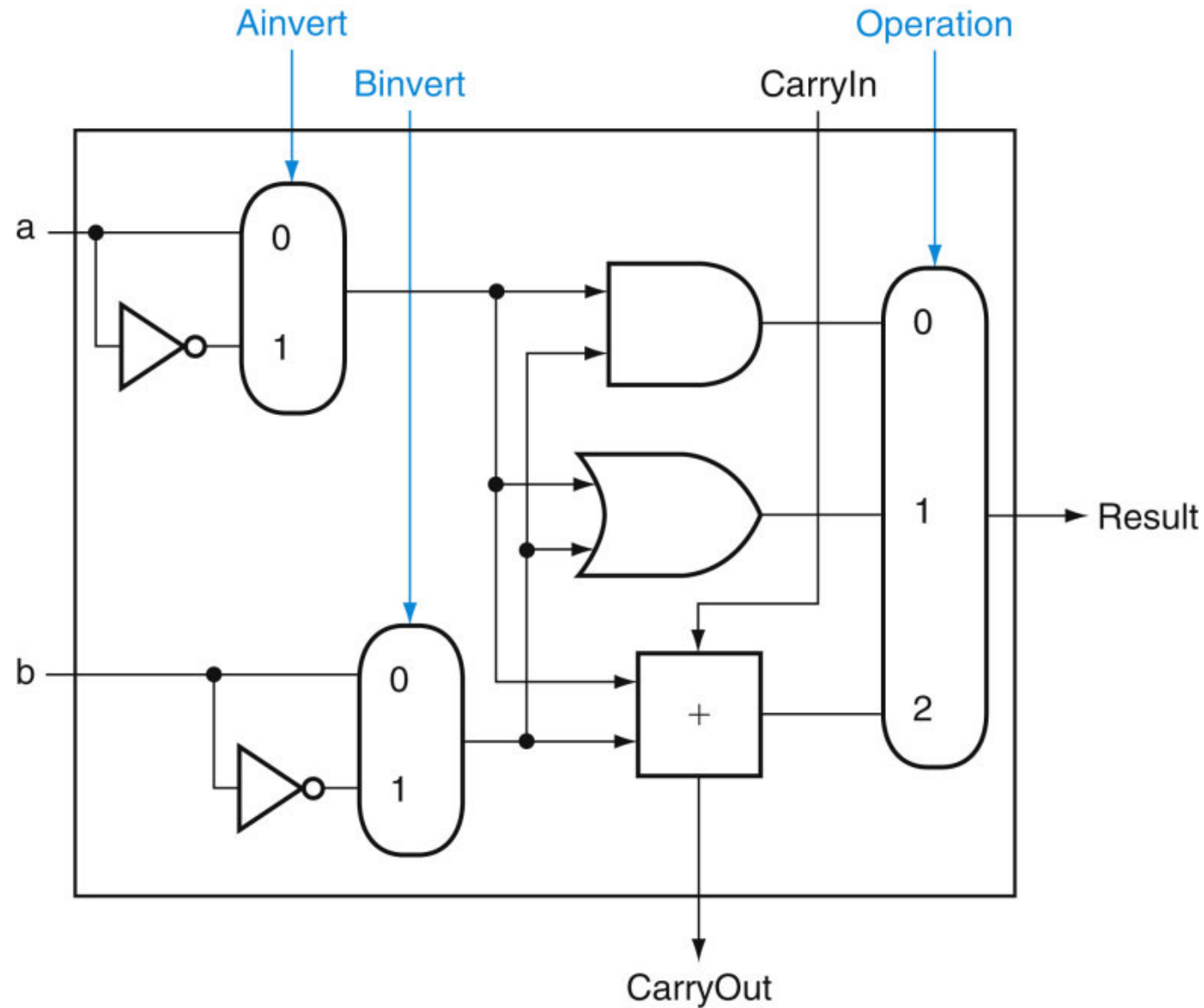
64-bit ALU



1-bit ALU with AND, OR, $a+b$, or $a+\bar{b}$



1-bit ALU with NOR

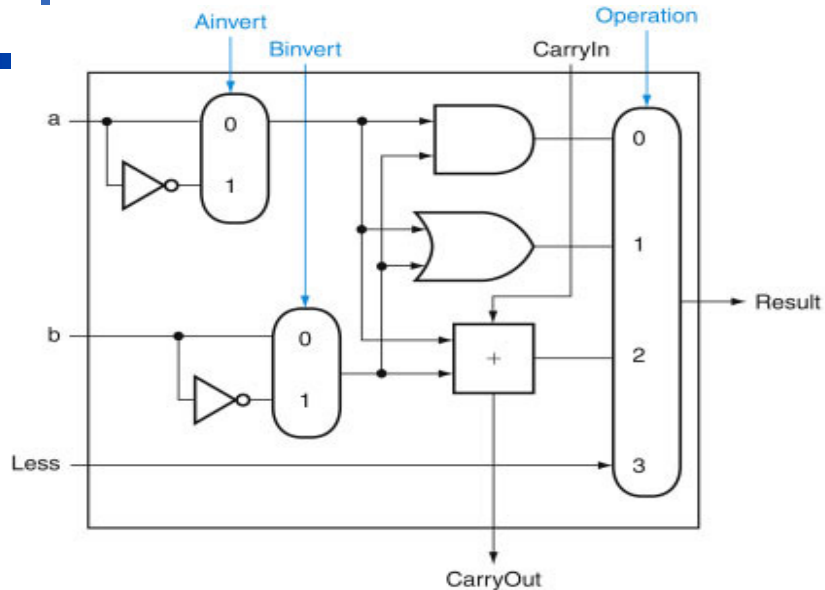


SLT

31	25 24	20 19	15 14	12 11	7 6	0
funct7	rs2	rs1	funct3	rd	opcode	
7	5	5	3	5	7	
0000000	src2	src1	ADD/ SLT / SLTU	dest	OP	
0000000	src2	src1	AND/OR/XOR	dest	OP	
0000000	src2	src1	SLL/SRL	dest	OP	
0100000	src2	src1	SUB/SRA	dest	OP	

ADD performs the addition of *rs1* and *rs2*. SUB performs the subtraction of *rs2* from *rs1*. Overflows are ignored and the low XLEN bits of results are written to the destination *rd*. **SLT** and **SLTU** perform signed and unsigned compares respectively, writing 1 to *rd* if *rs1* < *rs2*, 0 otherwise. Note,

ALU with SLT



上图：计算AND、OR、ADD和SUB的1位ALU

下图：计算最高位的1位ALU

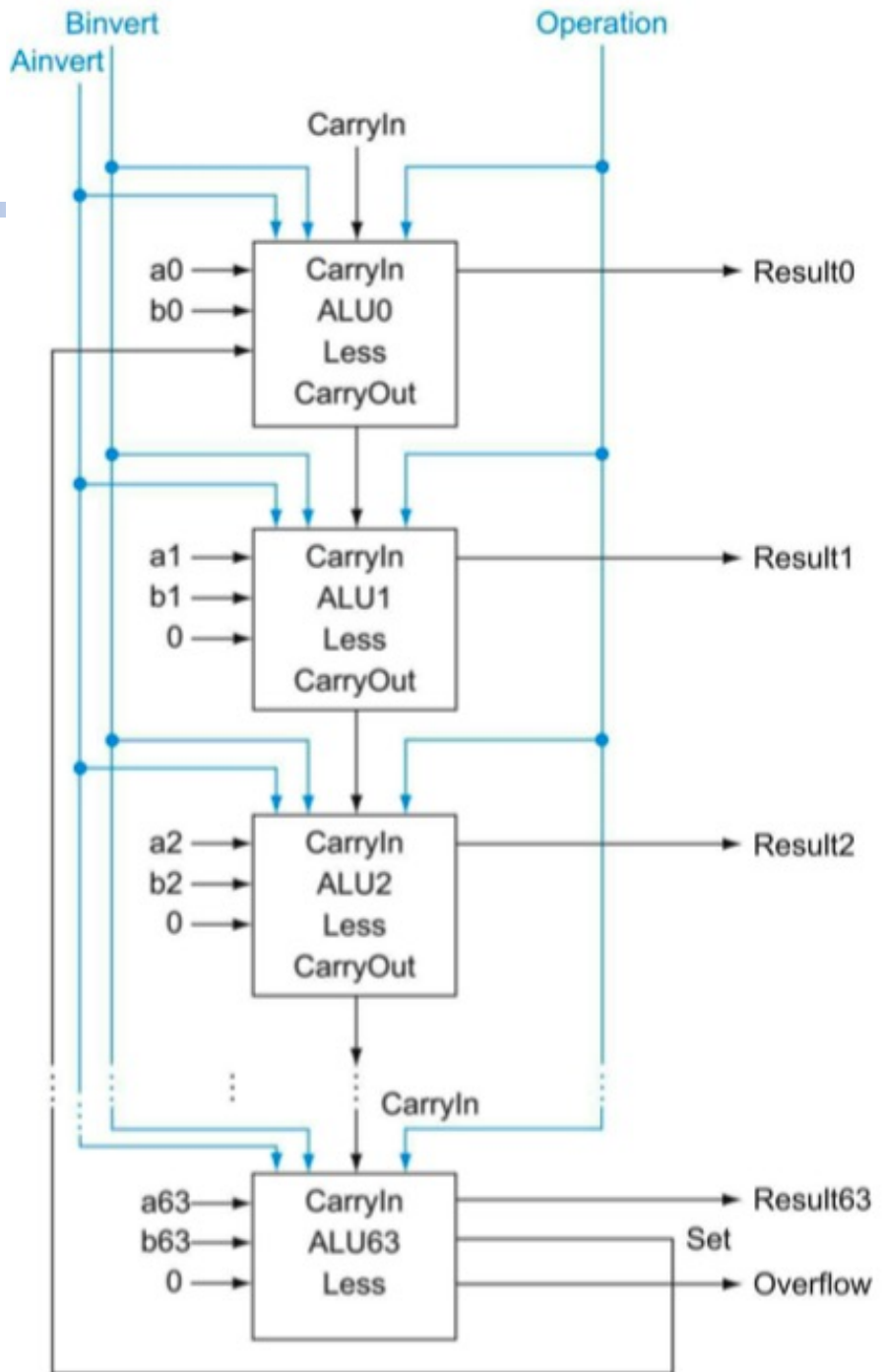
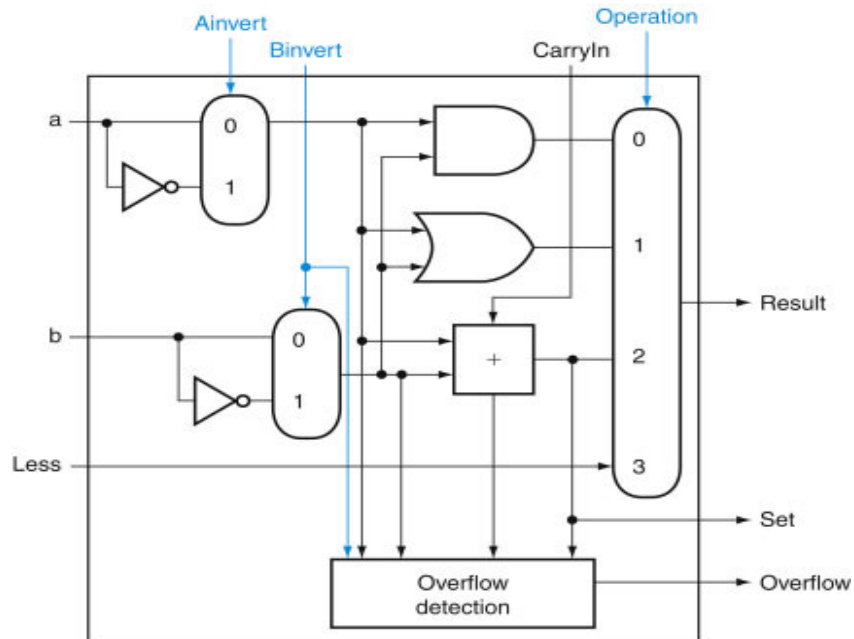


FIGURE A.5.11 A 64-bit ALU constructed from the

ALU with BEQ

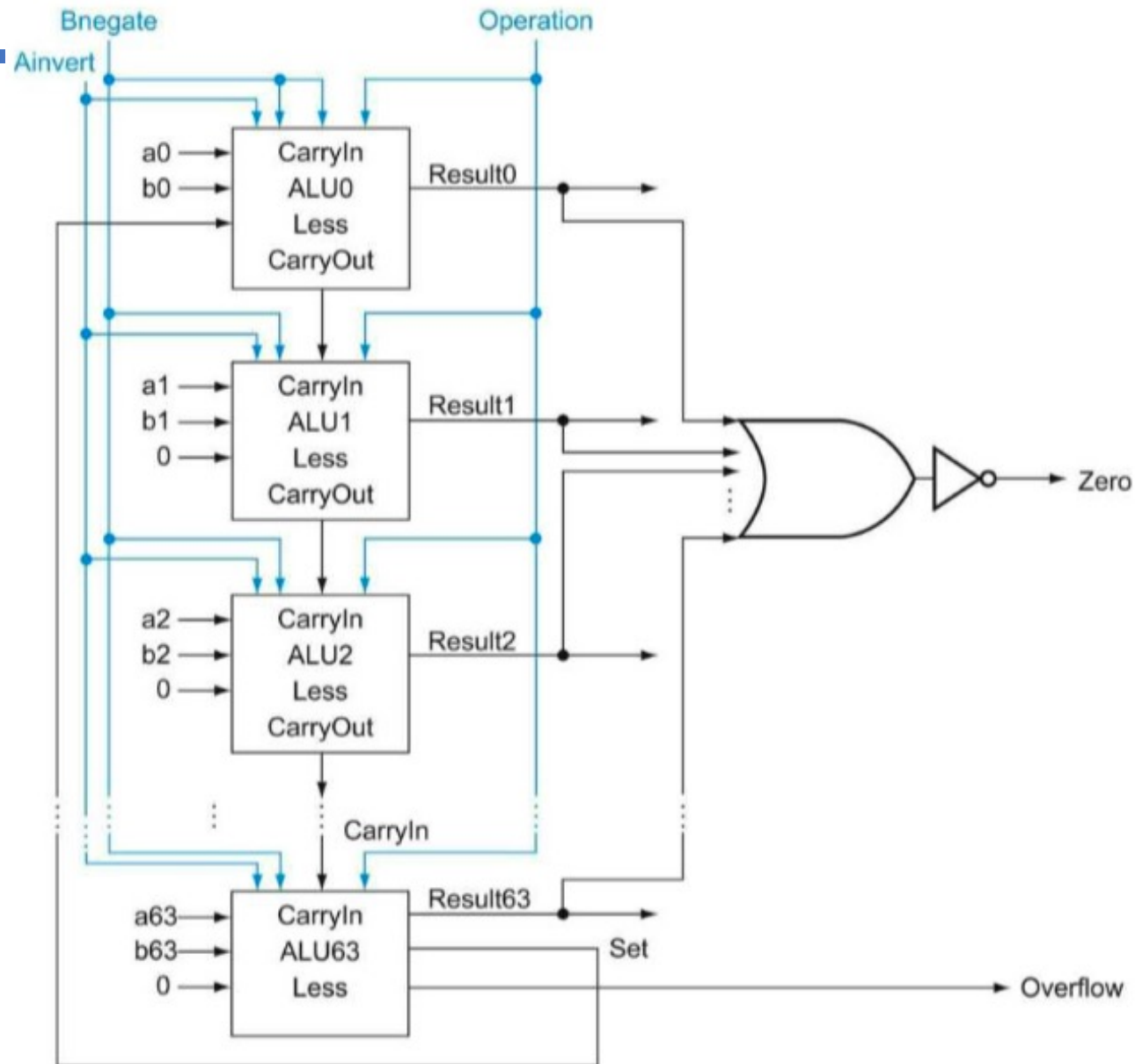
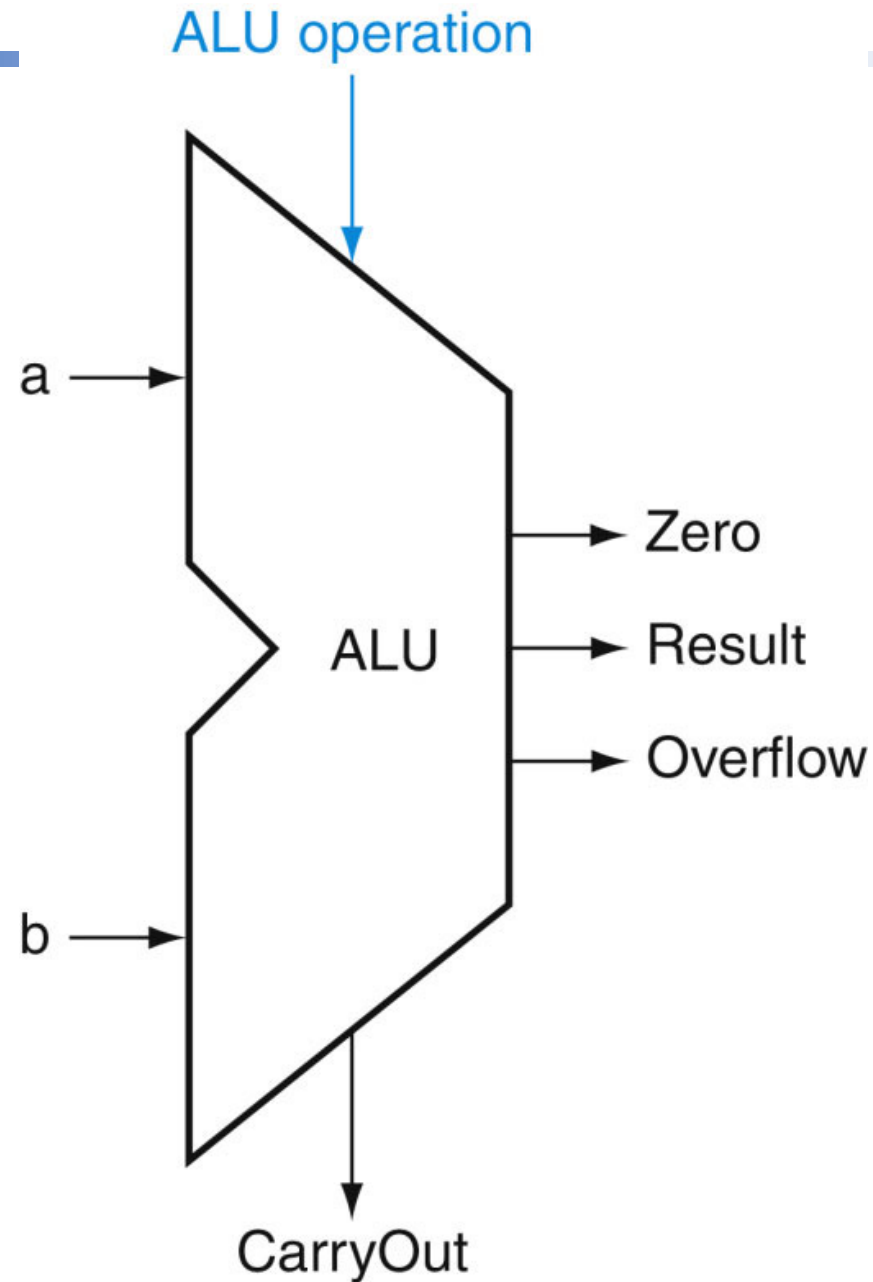


FIGURE A.5.12 The final 64-bit ALU.

The Symbol for ALU

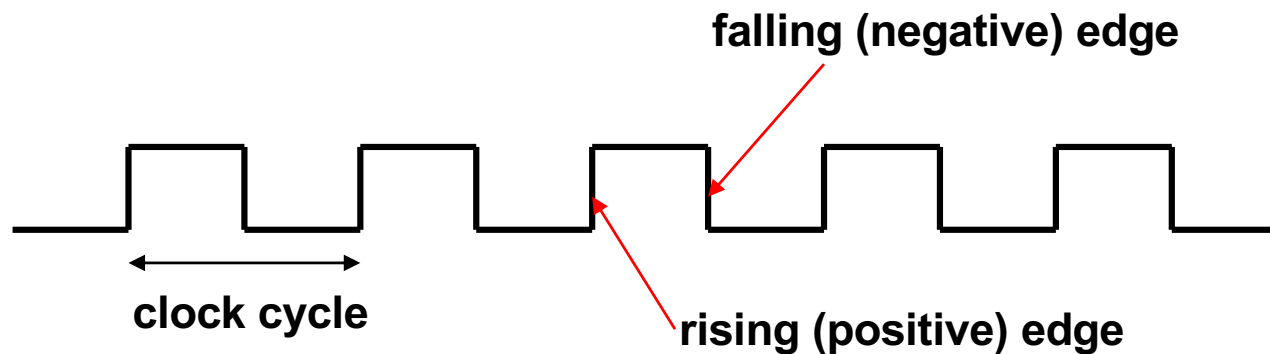


Exercise

- Suppose you wanted to add the operation NOT (a AND b), called NAND. How could the ALU change to support it?

Clocking Methodologies

- Clocking methodology defines when signals can be read and when they can be written



clock rate = $1/(\text{clock cycle})$

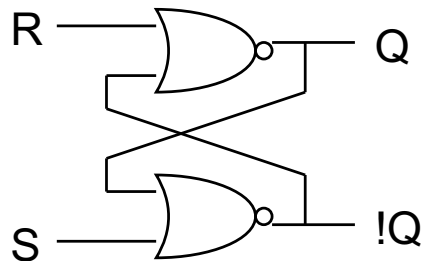
e.g., 10 nsec clock cycle = 100 MHz clock rate

1 nsec clock cycle = 1 GHz clock rate

- State element design choices
 - level sensitive latch
 - master-slave and edge-triggered flipflops

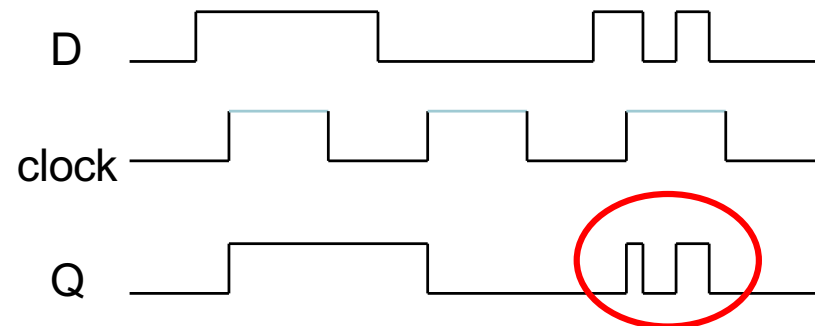
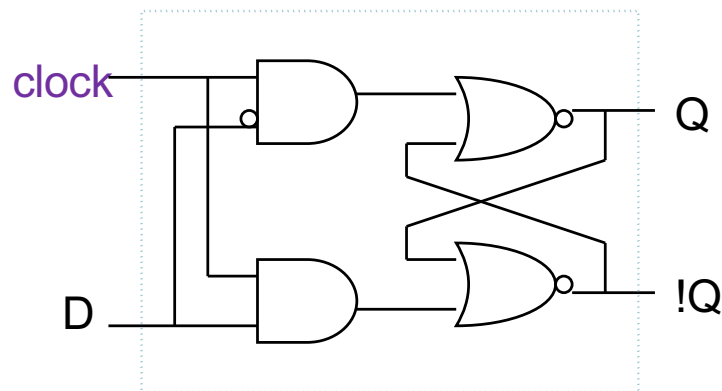
State Elements

■ Set-reset latch



R	S	$Q(t+1)$	$!Q(t+1)$
1	0	0	1
0	1	1	0
0	0	$Q(t)$	$!Q(t)$
1	1	X	X

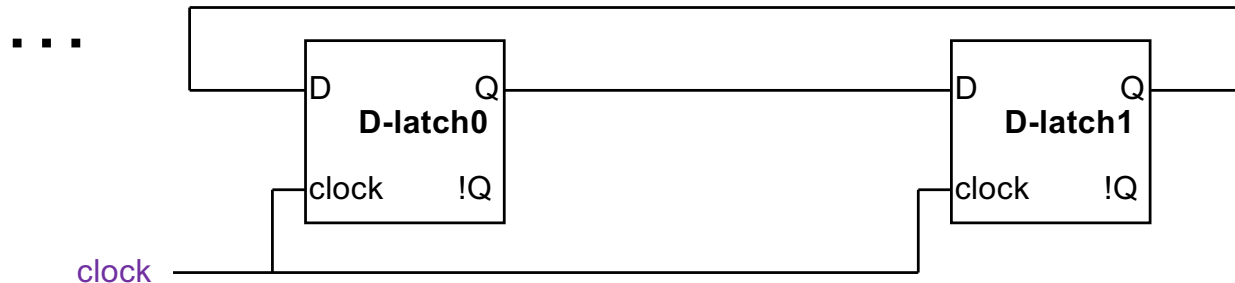
□ Level sensitive D latch



- latch is **transparent** when clock is high (copies input to output)

Two-Sided Clock Constraint

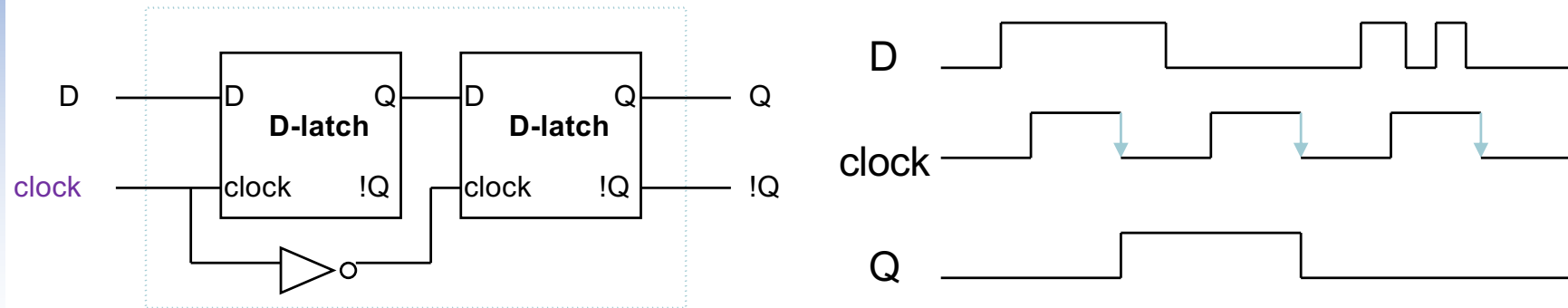
- Race problem with latch based design



- Consider the case when D-latch0 holds a 0 and D-latch1 holds a 1 and you want to transfer the contents of D-latch0 to D-latch1 and vica versa
 - must have the clock high long enough for the transfer to take place
 - must not leave the clock high so long that the transferred data is copied back into the original latch
- **Two-sided** clock constraint

State Elements, con't

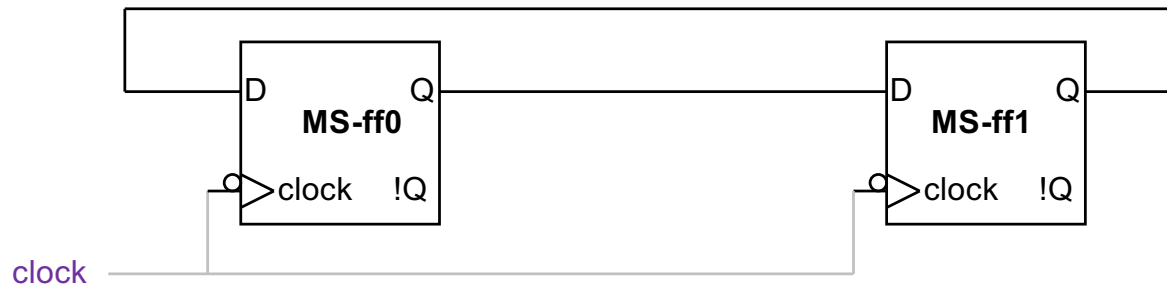
- Solution is to use flipflops that change state (Q) only on **clock edge** (master-slave)



- master (first D-latch) copies the input when the clock is high (the slave (second D-latch) is locked in its memory state and the output does not change)
- slave copies the master **when the clock goes low** (the master is now locked in its memory state so changes at the input are not loaded into the master D-latch)

One-Sided Clock Constraint

- Master-slave (edge-triggered) flipflops removes one of the clock constraints



- Consider the case when MS-ff0 holds a 0 and MS-ff1 holds a 1 and you want to transfer the contents of MS-ff0 to MS-ff1 and vica versa
 - must have the clock cycle time long enough to accommodate the worst case delay path
- **One-sided** clock constraint

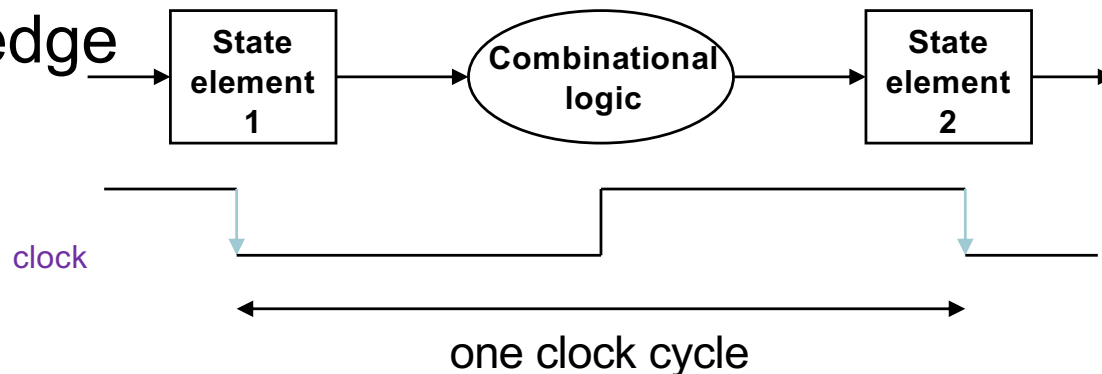
Latches vs Flipflops

- Output is equal to the stored value inside the element
- Change of state (value) is based on the clock
 - Latches: output changes whenever the inputs change and the clock is asserted (level sensitive methodology)
 - Two-sided timing constraint
 - Flip-flop: output changes only on a clock edge (edge-triggered methodology)
 - One-sided timing constraint

A clocking methodology defines when signals can be read and written – wouldn't want to read a signal at the same time it was being written

Our Implementation

- An edge-triggered methodology, typical execution
 - read contents of some state elements (combinational activity, so no clock control signal needed)
 - send values through some combinational logic
 - write results to one or more state elements on clock edge



- Assumes state elements are written on every clock cycle; if not, need explicit write control signal
 - write occurs only when **both** the write control is asserted and the clock edge occurs

Our Implementation

- The clock cycle must still be long enough so that the input values are stable when the active clock

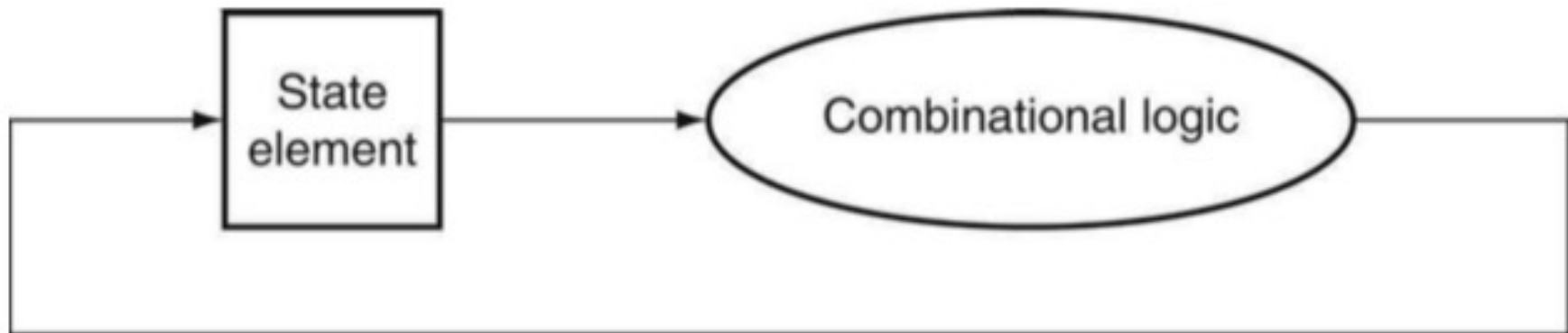
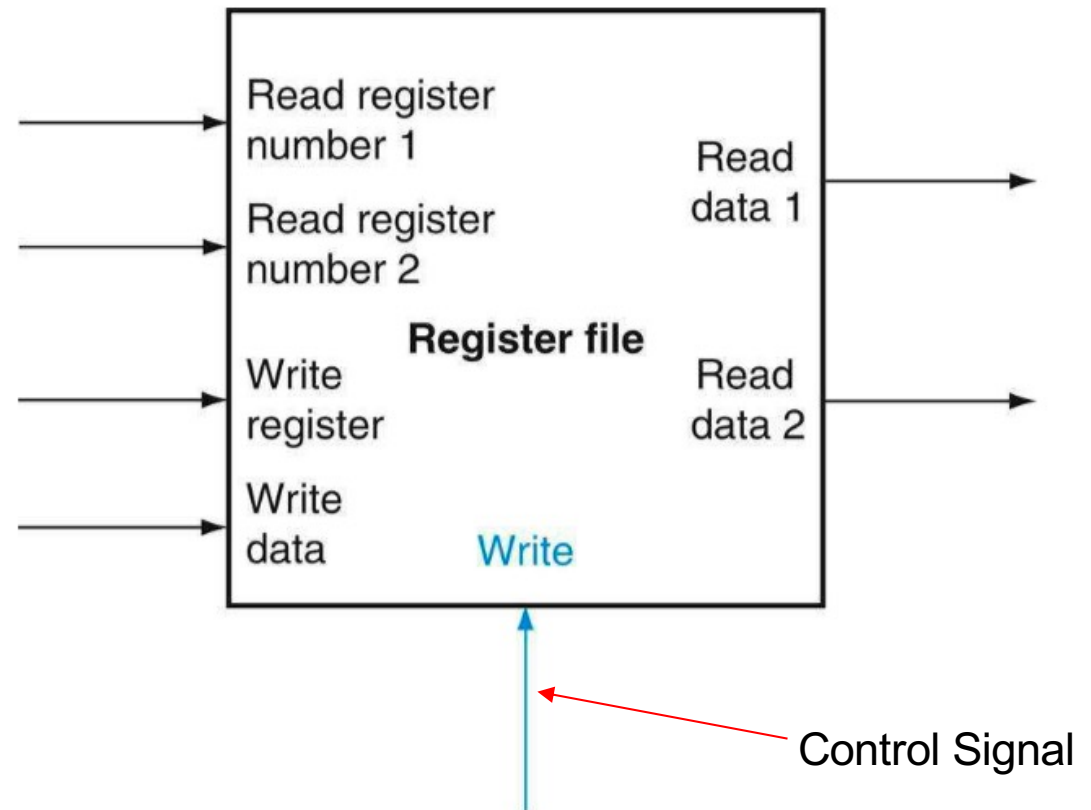


FIGURE A.7.3 An edge-triggered methodology allows a state element to be read and written in the same clock cycle without creating a race that could lead to undetermined data values.

Register File

- A register file consists of a set of registers that can be read and written by supplying a register number to be accessed
 - a decoder for each read or write port
 - an array of registers



Register File: read ports

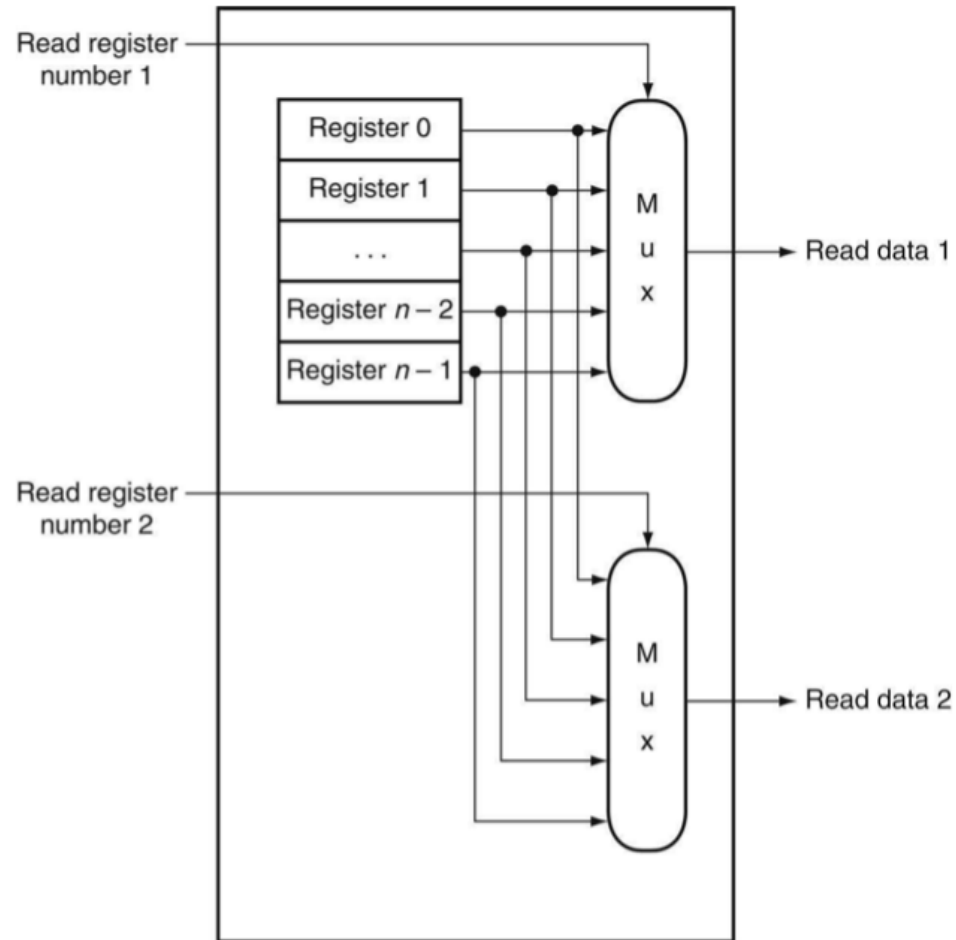


FIGURE A.8.8 The implementation of two read ports for a register file with n registers can be done with a pair of n -to-1 multiplexors, each 64 bits wide.

Register File: write port

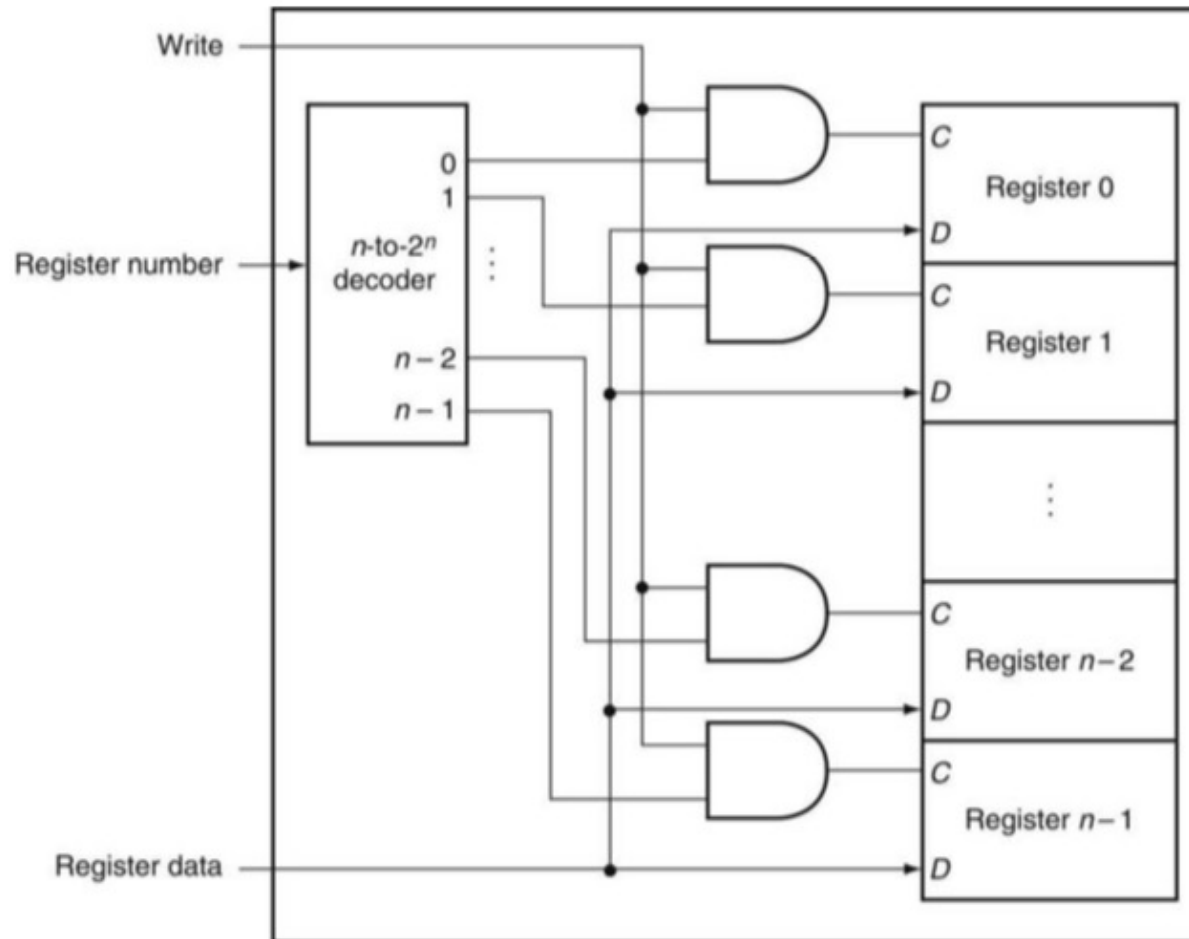


FIGURE A.8.9 The write port for a register file is implemented with a decoder that is used with the write signal to generate the C input to the registers.